

# Reinforcement Learning

## Final Project - 2026 B

### *Deep Reinforcement Learning from Pixels in MiniGrid*

This final project consolidates the practical and theoretical material of the course into one project on deep reinforcement learning from raw images. You will solve two custom MiniGrid environments with deep RL agents implemented from scratch: at least one value-based method (e.g. DQN), one policy-based method (e.g. REINFORCE), and one actor-critic method (e.g. PPO). In every case the agent sees only the rendered RGB image of the grid and must learn to act directly from pixels.

**A note on this assignment.** The sections below are topics to engage with, not a checklist. Many design choices (network architecture, observation preprocessing, reward shaping, what to measure, how to present results, where to invest extra effort) are deliberately left to you. The mission in ComplexEnv is genuinely hard: partial progress that is clearly analyzed is valued, so read the grading note in section 6 before you start.

## **1. Environments**

- Built on the MiniGrid framework: a 2D grid-world with a triangular agent and a discrete action space.
- Both environments are fully defined in the template notebook (layout, dynamics, action space, helper methods, editable lines), which is the single source of truth. Do not import them from the standard MiniGrid library.
- Both return a pixel observation only: the raw rendered RGB image of the grid. There is no symbolic or tabular state for the agent to read.

MiniGrid documentation: <https://minigrid.farama.org/environments/minigrid/>

### **1.1 SimpleRoomEnv (sanity check)**

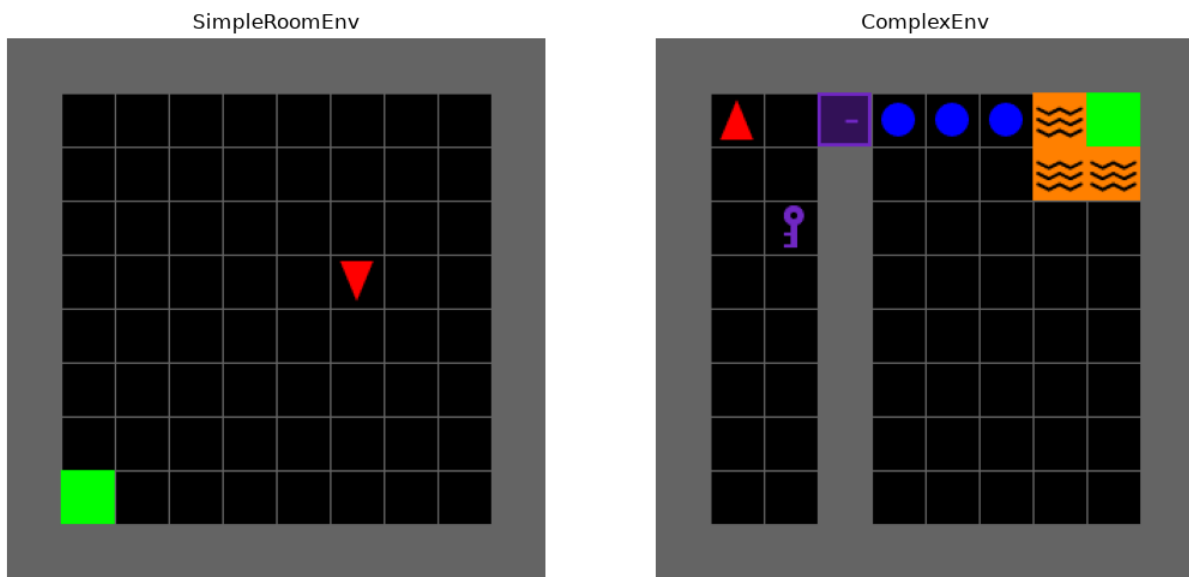
- An empty 10x10 room: the agent starts at a random interior cell with a random facing and must reach a green goal in a corner under a sparse reward (+1 on the goal, 0 otherwise).
- Purpose: a pass/fail sanity check for your image-to-action pipeline. If an agent cannot solve this, the problem is in your preprocessing, network, hyperparameters, or training loop, not in the hard task.
- You may set the episode length (max\_steps) at instantiation; nothing else may change.

### **1.2 ComplexEnv (the task)**

Two rooms split by a partition wall: a left key-room and a right room holding water balls and a lava-ringed goal. The mission has five ordered stages, key then door then water then lava then goal:

1. Pick up the key in the left room.

2. Toggle the locked door open, then drop the key (only one object can be carried at a time).
  3. Pick up a water ball in the right room.
  4. Face a lava tile and toggle to extinguish it: the tile clears and the ball is consumed. One ball per tile, so water must be ferried repeatedly.
  5. Step onto the goal in the lava-ringed corner. Stepping on un-extinguished lava is lethal and ends the episode.
- The layout is randomized per episode (key position, door row, goal corner, water cluster corner).
  - Sparse reward: +1 on the goal, 0 otherwise; a lava death ends the episode with 0.
  - The full specification (layout, dynamics, helper methods) is given in the notebook.



*Initial states of the two environments. Red triangle: agent. Green: goal. Purple: key and its locked door. Blue: water balls. Orange: lethal lava ringing the goal.*

### 1.3 Observations: pixels only

- Both environments return the full-grid RGB render as the observation: a uint8 array of shape (H·tile x W·tile x 3). There is no symbolic state available to the agent.
- You must turn this image into a network input yourself, inside a gym.ObservationWrapper (for example resize, channel-first, optional grayscale, frame-stacking).
- **Hard rule: the agent may consume pixels only.** The environment helper getters and object positions (below) are for reward-shaping event detection and for evaluation / EDA logging. They must never be fed to the network.

## 1.4 Actions

The action space is the standard MiniGrid Discrete(7). Carry semantics are real: drop is a first-class action, and toggle both opens the locked door (while holding the key) and extinguishes a lava tile while the agent is carrying water.

| ID | Name    | Effect                                                                                                |
|----|---------|-------------------------------------------------------------------------------------------------------|
| 0  | left    | turn 90 degrees left (no move)                                                                        |
| 1  | right   | turn 90 degrees right (no move)                                                                       |
| 2  | forward | move one cell forward                                                                                 |
| 3  | pickup  | pick up the object directly in front                                                                  |
| 4  | drop    | drop the carried object into the cell in front                                                        |
| 5  | toggle  | open the locked door while holding the key, or extinguish the lava tile in front while carrying water |
| 6  | done    | unused here (no-op)                                                                                   |

- You may restrict the agent to the subset of actions a task actually needs via a `gym.ActionWrapper` (for example dropping the no-op done action). State which actions you keep for each task.

## 1.5 Reward shaping

- The base reward is sparse, so reward shaping is allowed and encouraged for `ComplexEnv`. Implement any shaping in a wrapper.
- You may use the environment getters to detect events to reward or penalize; the full getter list is documented in the notebook.
- **Forbidden: distance- and geometry-based signals.** No reward may depend on how close the agent is to the key, door, water, lava, or goal (including any tracked minimum distance). This trivializes the task and leaks symbolic state into a pixel agent.
- Think about what your shaping actually encourages, and what it might encourage unintentionally. The notebook ships example wrappers as a starting point; you may extend them or write your own.

## 1.6 Template notebook

- The template notebook defines both environments, three example wrappers (grayscale, event shaping, action subset), an environment explorer that lets you drive an env by hand and watch the getters and shaped reward update, and a 'Your Code Below' divider. It is your starting point.

- **The environment explorer is optional.** To use it, keep env\_explorer.py in the same folder as the notebook, or on Colab upload it through the Files panel.
- **Colab link:**  
<https://colab.research.google.com/drive/1yLPGXiVCxYczlPRluRGLjcZKIixq1wVk?usp=sharing>
- **The environment classes are fixed.** You may override max\_steps at construction and nothing else. Do all preprocessing, reward shaping, and action restriction in wrappers, and place your agents below the divider.
- **Use a GPU.** Deep learning trains much faster on a GPU than on CPU. On Colab, enable a GPU runtime (Runtime, then Change runtime type). Consider checkpointing long runs so you can resume them.

## 2. Analysis & Design

For each environment, engage with the topics below in both your notebook and your report. You decide how deeply to treat each one and which trade-offs to emphasize.

### 2.1 Understanding the environments

- Characterize each environment as an MDP: episodic or continuing, the action space, and, crucially, the observation as a high-dimensional image rather than a small discrete index. Discuss observability: the full grid is visible in the image, but the agent must infer task state (what it carries, whether the door is open, which lava remains) from pixels.
- Estimate the size of each environment's state space: the distinct configurations the environment can actually produce, not the space of all possible RGB images. Sizing it as if you were building a Q-table shows why a tabular method is impractical here and motivates deep RL with function approximation.
- Point out the differences that matter for your approach: horizon length, reward sparsity, the role of shaping, the cost of a lethal mistake, and which stages are recoverable (dropping the key or wasting water is recoverable, but stepping on un-extinguished lava is lethal).

### 2.2 Observation preprocessing & state representation

- Describe your image pipeline: input resolution, resizing, color versus grayscale, channel order, normalization, and whether you stack frames and why.
- Explain what information you keep and what you discard, and state the resulting network input shape, including whether you keep color or convert to grayscale.
- State which of the seven actions your policy actually uses for each task and which you drop.

### 2.3 Network architecture

- Describe each network architecture in full (input shape, layer sizes, output head(s), and any key design choices) for every algorithm you implement.

## 2.4 Algorithms

Implement at least one algorithm from each family, from scratch, and run each one on both environments:

- **Value-based** (e.g. DQN).
- **Policy-based** (e.g. REINFORCE).
- **Actor-critic** (e.g. PPO).
- Discuss the advantages and disadvantages of each in the context of this image-based, sparse-reward task: sample efficiency, stability, variance, and sensitivity to reward shaping.
- Efficiency matters: prefer solutions that converge in fewer environment steps and act decisively at inference.

## 2.5 Exploration & hyperparameters

- Choose and justify your exploration strategy, for example an epsilon-greedy schedule for value-based methods or an entropy bonus for policy-gradient and actor-critic methods.
- Choose the learning rate, discount factor gamma, batch size, replay or rollout settings, and target-network update rate deliberately; they change results dramatically.
- Document every hyperparameter so a reader can reproduce your runs.

## 3. Training Results & Analysis

For each algorithm and each environment, show how training progresses with graphs that use the training episode on the x-axis. Rolling averages are fine. Plot at least:

- Reward / return per episode.
- Number of steps taken per episode.
- Success rate per episode (whether the agent reached the goal).
- **Cumulative environment steps versus episode.** A running total of the environment steps consumed as episodes accumulate. Teams may pick very different settings (for example max\_steps 1000 over 100 episodes, or max\_steps 100 over 2000 episodes), so this curve places every run on a common budget axis, the total environment steps. Include it for both environments.

**Progress on ComplexEnv.** ComplexEnv is hard and may not be fully solved, so a single success/fail rate is not enough on its own. Show how far the agent gets through the five stages (key, door, water, lava, goal) and what it has learned even when it does not reach the goal, so that partial progress is visible and can be credited. How you measure and present this is left to you.

## 4. Inference Results

- After training, evaluate each learned policy greedily (no exploration) over several evaluation episodes on fresh seeds.

- For each algorithm in each environment, report the average return, the average number of steps, and the success rate. For ComplexEnv, also report how far the agent gets through the five stages, so a competent-but-imperfect agent is measured fairly.
- Present the results so the comparison across the three algorithms and the two environments is clear, for example as a table and/or a grouped bar chart.

## 5. Discussion

- Discuss the strengths and weaknesses of each algorithm family on each environment, drawing on your diagnostics: which learned fastest, which was most stable, which got furthest on ComplexEnv, and why.
- Discuss what your reward shaping helped with and what it may have encouraged unintentionally.
- In a dedicated notebook cell, clearly state the best training and inference settings you found.

## 6. Note on Grading

Following these instructions thoroughly is the baseline for a strong grade. A perfect score requires going beyond the requirements: a well-motivated extension, a deeper analysis, or an insight that reveals genuine understanding of the material. The quality of that extra effort, not just its presence, is part of what we evaluate.

**ComplexEnv is hard, and you are not penalized for failing to fully solve it if your results clearly show how far each agent got and what it learned.** A thoughtful analysis of a partial solution can score higher than an unexplained full one. Reaching convergence in fewer total environment steps is a small competition par.

## 7. Rules & Constraints

1. Implement every algorithm yourself. Existing reinforcement-learning libraries (for example Stable-Baselines3, RLlib, Tianshou, CleanRL) are not allowed. Use PyTorch or TensorFlow for the neural network only.
2. Pixel observations only: the agent network may take the image as input and nothing else. The env getters and positions are for shaping-event detection and for evaluation / EDA logging, never as agent inputs, and no reward may use distance or geometry.
3. The environment classes are fixed: you may override max\_steps and nothing else. Implement all preprocessing, reward shaping, and action restriction in wrappers, and place your own code below the 'Your Code Below' divider.
4. Include the required training graphs (including the cumulative-steps curve) and your ComplexEnv progress analysis even if you do not fully solve an environment.
5. Include short video clips of the agent partway through training and after it has converged (in the notebook).
6. Keep the notebook clean, with well-separated code and text cells. A messy notebook lowers your grade.

7. Seed Python, NumPy, and your DL framework, and document every hyperparameter so your runs are reproducible.
8. Do not modify the notebook after the deadline; a re-run or edited notebook may be disqualified.

## 8. Report

**The report is the primary deliverable and must be fully self-contained.** Write it as a professional scientific document from which a reader can understand your entire methodology and all of your results without opening the notebook or the code.

- Include every graph, table, numerical result, and discussion point the assignment asks for directly in the report itself.
- Name the file report\_ID1\_ID2.pdf, using both partners' ID numbers.
- Put the project title and both partners' full names and IDs at the top.
- Keep the report to 12 pages or fewer.

## 9. Submission

Submit the following files to the submission box:

1. details.txt: the link to your notebook and your partner details (template below).
2. report\_ID1\_ID2.pdf: your final report.
3. explainer.txt (optional): how to run your notebook, plus anything else a reader should know.

### details.txt template:

link to the notebook:  
<https://colab.research.google.com/drive/folders/#####?usp=sharing>

Full name student number 1: Israel Israeli  
ID student number 1: 123456789

Full name student number 2: Israel Israeli  
ID student number 2: 123456789

- The notebook must be submitted as a Google Colab link the course team can run, and it must already contain all of the outputs from training and inference.

## 10. Team Size

The project is carried out in pairs. Submitting individually, without prior approval, results in a grade deduction.

## 11. Academic Integrity

- All submitted work must be your team's own. Copying code or answers from other students or teams, or sharing your own work, violates the university honor code, and you are responsible for protecting your work from copying.

- You may build on existing code (your own or external) if you cite it clearly and state what you changed and added. Unattributed use of others' work is a breach of academic integrity.
- Suspected violations are reported to the head of department and the university authorities.